

TP Final: Coding Agent Avanzado

Evolucionar el coding agent construido en clase, sin frameworks de orquestación.

Objetivo

A partir del coding agent desarrollado en el TP en clase, construir un sistema de agentes de IA para resolver tareas de código sobre un lenguaje, framework o ecosistema técnico elegido por el grupo y aplicado a un caso de uso concreto.

El sistema debe combinar herramientas locales, RAG sobre documentación técnica, memoria persistente del proyecto, subagentes especializados, políticas de seguridad y observabilidad.

No se permite utilizar frameworks de orquestación de agentes como LangChain, LangGraph, CrewAI, AutoGen ni similares. Se puede usar cualquier lenguaje, API de LLM y librerías puntuales para embeddings, almacenamiento vectorial, búsqueda web, observabilidad o CLI.

Punto de partida

El trabajo se construye a partir del coding agent realizado en el TP en clase. Debe conservar el harness y las herramientas base (lectura, escritura, ejecución de comandos, exploración de archivos y búsqueda web), pero evolucionar hacia una arquitectura de agentes más completa.

Arquitectura de agentes

El sistema debe contar con un agente principal que recibe la tarea del usuario, mantiene el estado general y coordina el trabajo de los subagentes. El agente principal puede ejecutar tools directamente si el grupo lo considera necesario.

Además, se deben implementar los siguientes subagentes. Cada uno debe tener una responsabilidad clara. No es necesario que todos tengan acceso a las mismas tools ni permisos.

Subagente	Responsabilidad
Explorer	Entiende el repositorio: estructura, arquitectura, dependencias, convenciones y archivos relevantes.
Researcher	Busca información en el RAG y, cuando sea necesario, en la web.
Implementer	Propone o realiza cambios de código a partir de los hallazgos disponibles.
Tester	Valida el resultado mediante checks definidos por el grupo: tests, build, lint, logs u otras verificaciones.
Reviewer	Revisa el diff o los cambios realizados y valida que respondan al pedido del usuario.

Estado compartido y memoria

El agente principal y los subagentes deben compartir un estado de tarea. Puede implementarse como un objeto en memoria, un JSON u otra estructura simple. Debe registrar, como mínimo, el pedido original, el avance, los resultados de los subagentes, las fuentes consultadas, los archivos modificados y las observaciones relevantes.

Además, el sistema debe contar con memoria persistente por proyecto, más allá del historial de una conversación. Puede guardar arquitectura detectada, archivos importantes, dependencias, comandos útiles, convenciones, decisiones tomadas, bugs investigados y resúmenes de sesiones previas.

RAG y búsqueda de información

El agente debe estar especializado en un lenguaje, framework o ecosistema técnico elegido por el grupo. Debe contar con un sistema de RAG sobre documentación, ejemplos, READMEs o proyectos de referencia de ese ecosistema.

- Implementar chunking, embeddings y almacenamiento vectorial.
- Recuperar contexto relevante antes de responder o tomar decisiones.
- Mostrar qué documentos o fragmentos recuperó y utilizó.
- Diferenciar entre información obtenida del repositorio, memoria, RAG, web e inferencias propias.
- Consultar primero el RAG. Cuando no exista evidencia suficiente, usar búsqueda web como fallback, priorizando documentación oficial y fuentes técnicas confiables.

Manejo de contexto y comportamiento del agente

El sistema debe implementar una estrategia para manejar conversaciones, tareas y proyectos extensos. Debe resumir información anterior, conservar decisiones relevantes y evitar enviar todo el repositorio o historial completo al modelo en cada turno.

También debe detectar cuándo está repitiendo acciones sin avanzar, por ejemplo al ejecutar varias veces un comando que produce el mismo error o al releer archivos sin obtener información nueva. En esos casos debe cambiar de estrategia, replanificar, detenerse o pedir ayuda.

El agente debe poder reconocer cuándo no tiene evidencia suficiente para continuar: pedidos ambiguos, falta de documentación, restricciones de permisos, errores no diagnosticados o cambios demasiado riesgosos. Debe explicar qué intentó, qué información falta y qué necesita para seguir.

Archivo de configuración y políticas del agente

El sistema debe leer una configuración, por ejemplo `agent.config.yaml` o `agent.config.json`. La configuración debe validarse antes de ejecutar cada tool call.

Política	Ejemplos
Lectura	Archivos o directorios que el agente no puede leer: <code>.env</code> , <code>secrets/**</code> , <code>**/*.pem</code> .
Escritura	Archivos o directorios que el agente no puede modificar: <code>.github/**</code> , archivos de lock u otros definidos por el grupo.
Comandos	Comandos prohibidos, como <code>rm -rf</code> o <code>git push</code> .
Aprobación	Comandos que requieren confirmación del usuario, por ejemplo <code>npm install</code> , <code>pip install</code> o <code>git commit</code> .

Ejemplo de configuración:

```
workspace: ./mi-proyecto
permissions:
  read:
    deny: [".env", "**/*.pem", "secrets/**"]
  write:
    deny: [".github/**", "package-lock.json"]
commands:
  deny: ["rm -rf", "git push"]
  require_approval: ["npm install", "pip install", "git commit"]
```

Observabilidad

El proyecto debe integrar Langfuse, LangSmith, Phoenix u otra herramienta equivalente. Debe utilizarse en al menos una de las pruebas entregadas.

Como mínimo, se deben registrar prompts, modelo utilizado, llamadas al LLM, tools invocadas, documentos recuperados, búsquedas web, iteraciones, errores, latencia, tokens, costo estimado y resultado final.

Caso de uso concreto

Cada grupo deberá definir un objetivo concreto para utilizar el agente sobre un repositorio, proyecto o ecosistema técnico elegido por ustedes. El objetivo debe producir un resultado verificable y no puede ser solamente "probar que el agente funciona".

Caso de uso	Resultado esperado
Analizar un repositorio desconocido	Generar un reporte de arquitectura, dependencias, riesgos y comandos relevantes.
Agregar una funcionalidad a un proyecto existente	Implementar una mejora concreta y documentar los cambios realizados.

¿Qué probar con el agente?

Cada grupo debe diseñar las pruebas de su sistema en relación con el caso de uso elegido. Las tareas deben permitir observar el trabajo coordinado de los subagentes y las capacidades incorporadas.

- Una tarea que requiera usar RAG y mostrar las fuentes recuperadas.
- Una tarea que requiera usar memoria del proyecto.
- Una tarea en la que el agente deba cambiar de estrategia, detenerse o pedir ayuda.
- Al menos una ejecución registrada en la herramienta de observabilidad elegida.

Extra opcional

Feature	Descripción
Sistema de plugins para tools	Permitir agregar nuevas tools sin modificar el núcleo del harness. Cada tool debe seguir una interfaz común: nombre, descripción, schema de parámetros, función de ejecución y políticas de permisos. El sistema debe descubrir y registrar automáticamente las tools habilitadas.

Entregables

1. Código completo y funcionando, construido a partir del coding agent del TP en clase.
2. README con instrucciones de instalación, configuración y ejecución.
3. Breve descripción del caso de uso elegido: repositorio o proyecto utilizado, objetivo concreto y criterio para considerar que fue cumplido.
4. Breve explicación de la arquitectura: rol del agente principal, rol de cada subagente y estructura del estado compartido.
5. Documentación de la base RAG: fuentes utilizadas, estrategia de chunking, embeddings y almacenamiento elegido.
6. Evidencia de al menos dos tareas ejecutadas sobre el caso de uso, incluyendo el output relevante, las fuentes recuperadas y una breve explicación de qué se observa en cada caso.
7. Capturas de pantalla de la herramienta de observabilidad mostrando al menos una traza completa de ejecución.
8. Reflexión breve: qué funcionó bien, qué falló, cuándo se detectaron loops o falta de evidencia y qué mejorarían del sistema.